AEROSPACE ENGINEERING · STUDENT PROJECT · 2026

Lambert Studio

A desktop application, written in Python and JavaScript, that finds a satellite's orbit from two ground-station sightings (Lambert's problem), plans orbital manoeuvres, predicts how an orbit drifts under real forces, and designs gravity-assist routes to the planets. Given only the planets and a launch period, its route optimiser finds the launch and flyby dates of Voyager, Cassini and Galileo to within days.



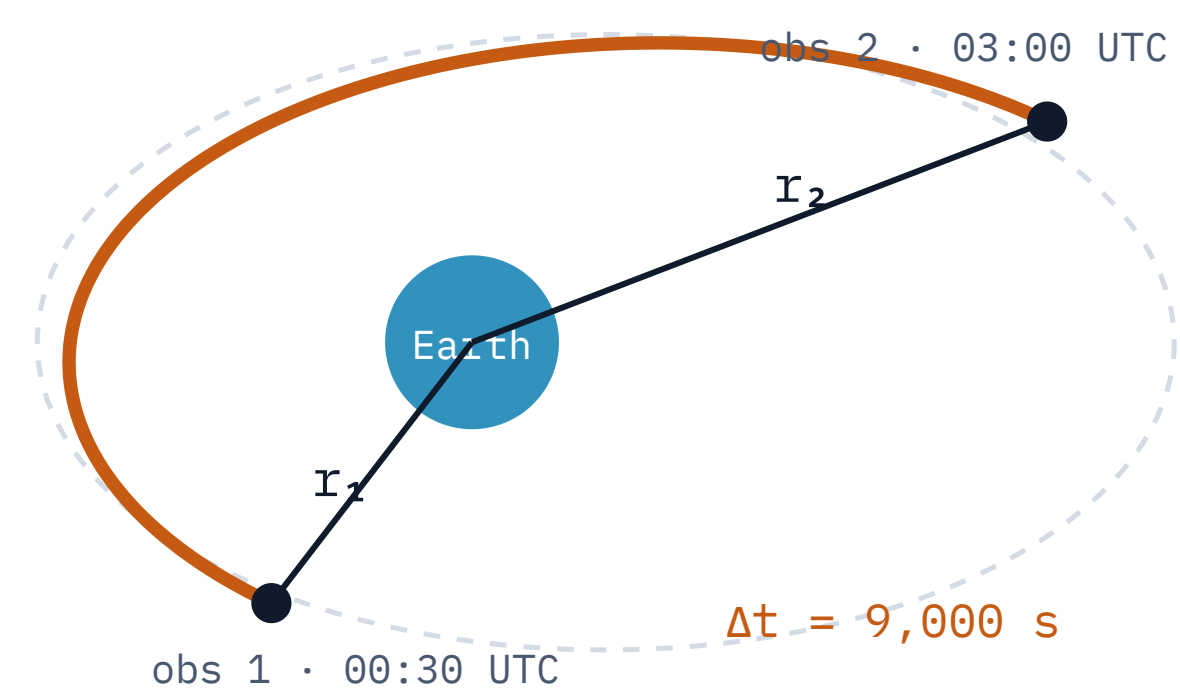
Mission playback, Cassini (1997). The route found by the optimiser: two Venus flybys, one Earth flyby, Jupiter, then Saturn. The telemetry panel tracks distance from the Sun, speed and Δv used; the timeline marks each event.

T+0

STARTING POINT

One assignment: find an orbit from two sightings

A ground station measures a satellite twice: azimuth, elevation and range, two and a half hours apart. From those six numbers, find the complete orbit. This is **Lambert's problem**, the same mathematics used to plan every interplanetary mission.



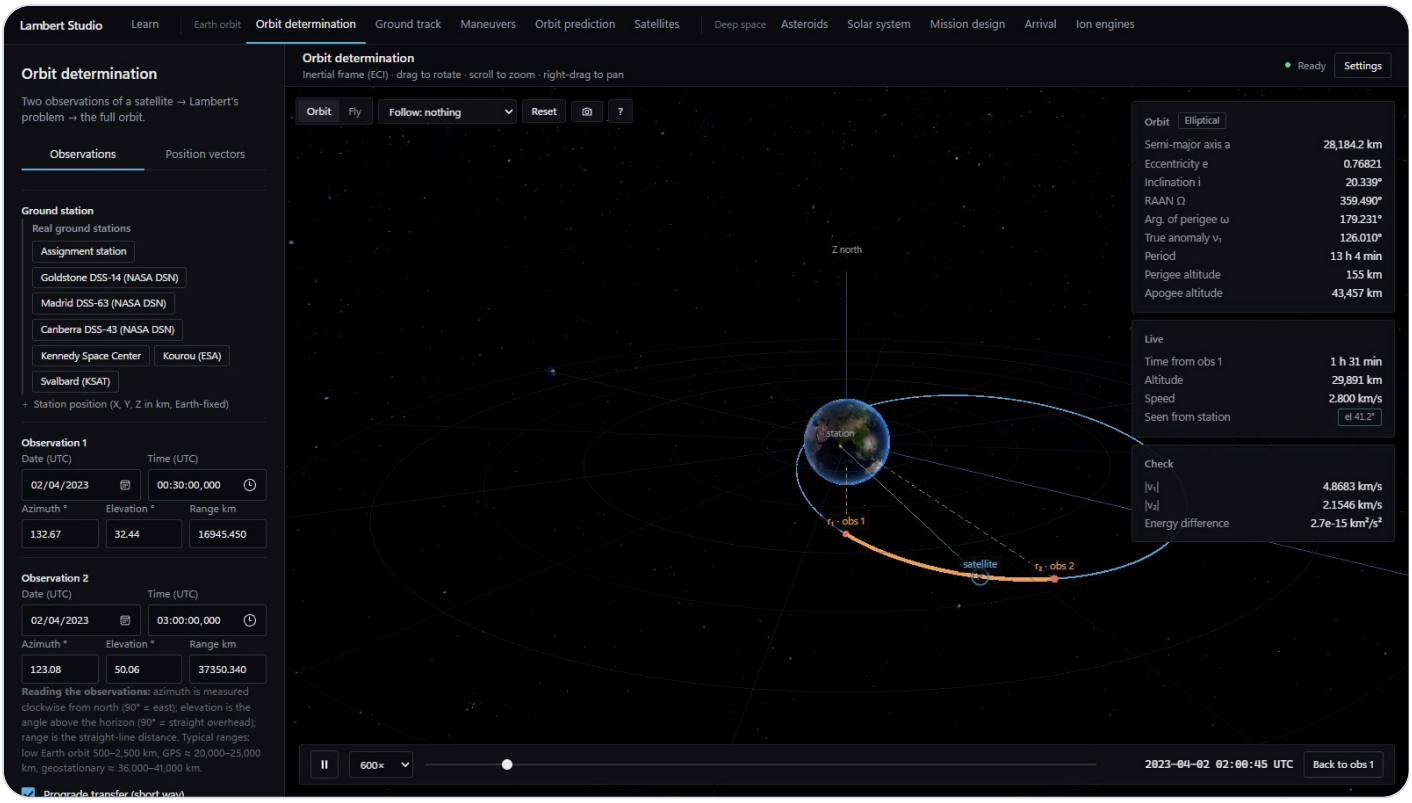
1. Convert each sighting (azimuth, elevation, range) from the station's horizon frame into Earth-centred inertial coordinates, using sidereal time.
2. Solve Lambert's problem with the universal-variable method (Curtis, Algorithm 5.2) to get the velocities.
3. Turn position and velocity into the six classical orbital elements.
4. Result for the assignment: an elliptical orbit with a = 28,184 km, e = 0.768, i = 20.34°, period 13 h 04 min.

T+1

WHAT IT DOES

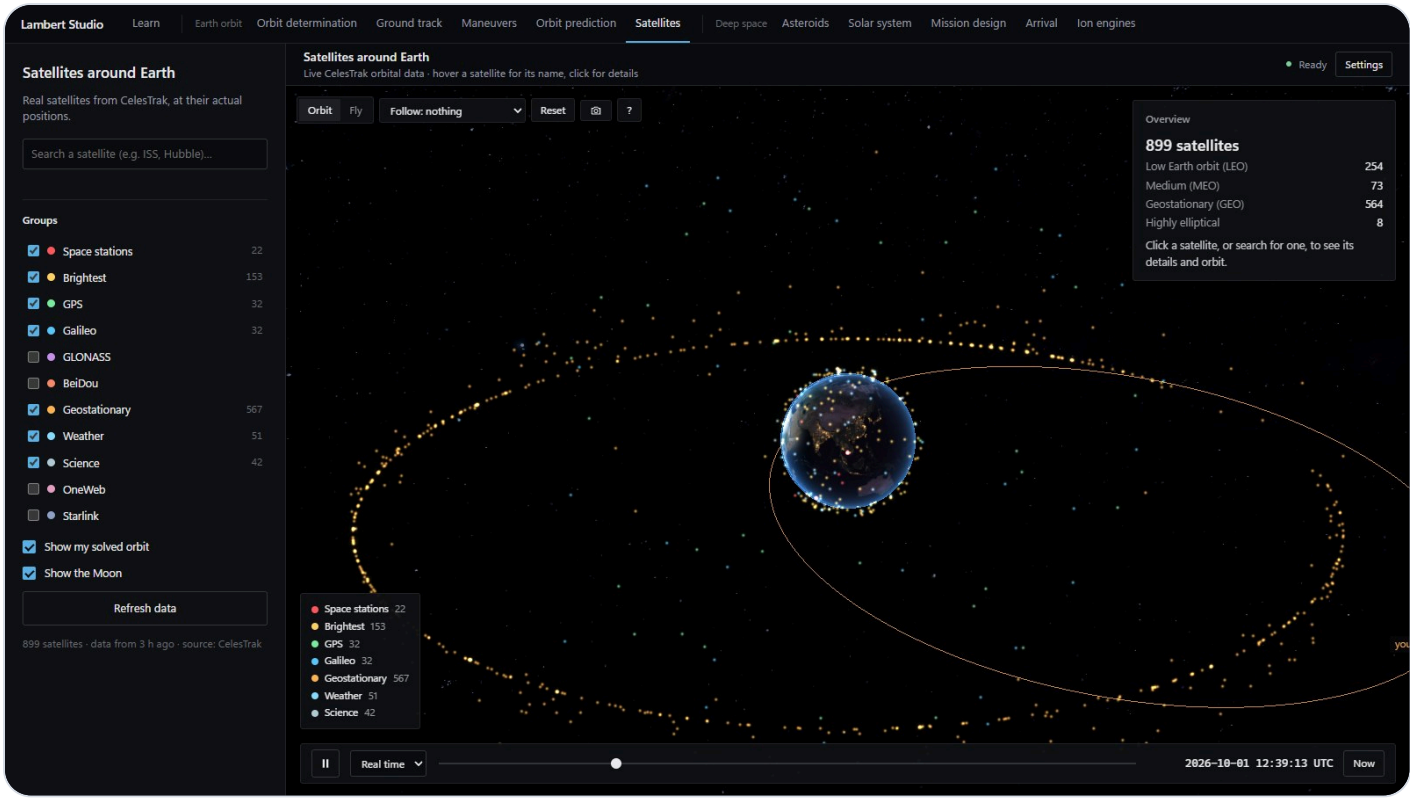
Eleven modules, one desktop app

Everything runs in one window, with a shared timeline, a free-flying camera in every 3D view, and recommended values drawn from real missions and facilities next to every input.



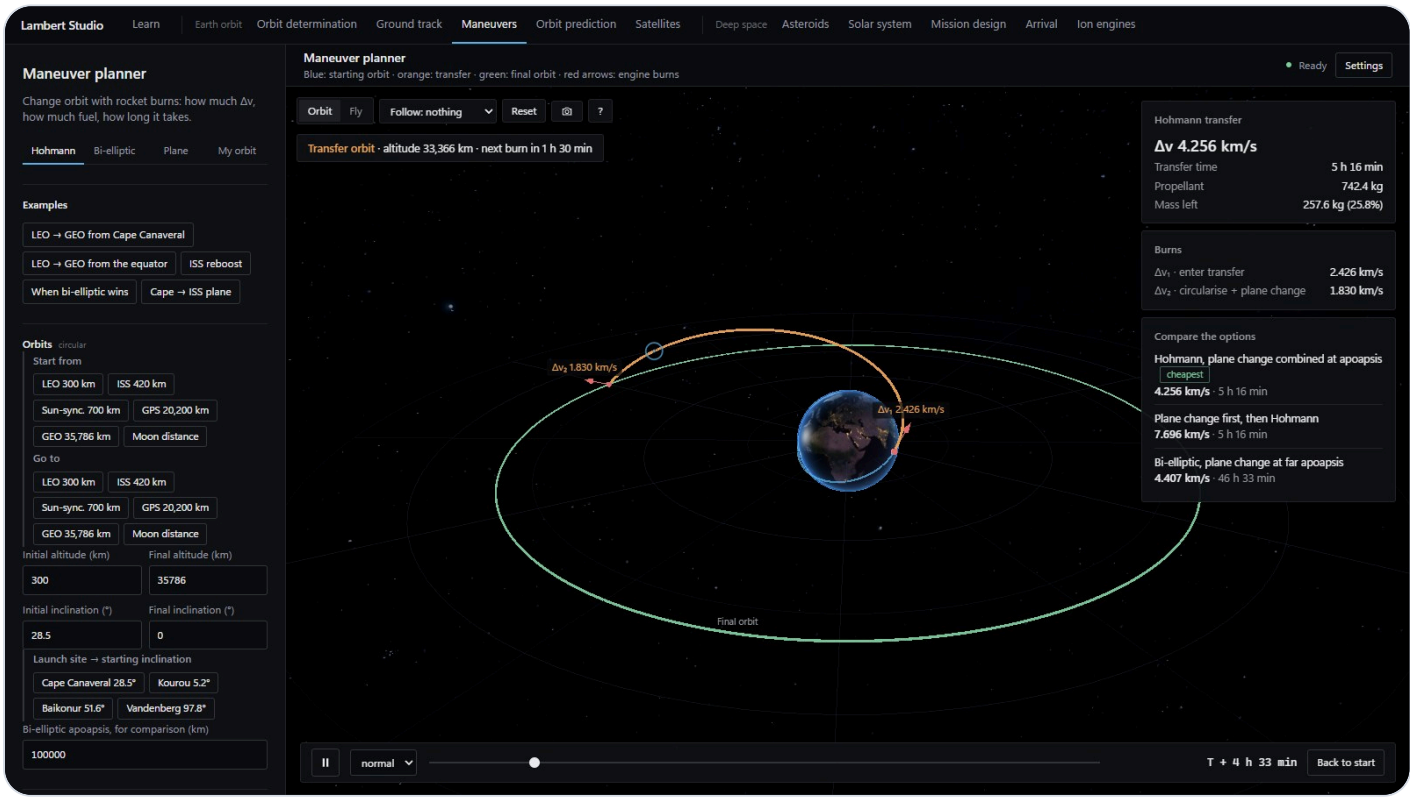
Orbit determination

The assignment, solved and drawn in 3D: day/night Earth from NASA imagery, the observed arc, apsides and the ground station's line of sight.



Real satellites

Live CelesTrak element sets: the ISS, GPS, Galileo, the geostationary ring, with search and per-satellite detail.



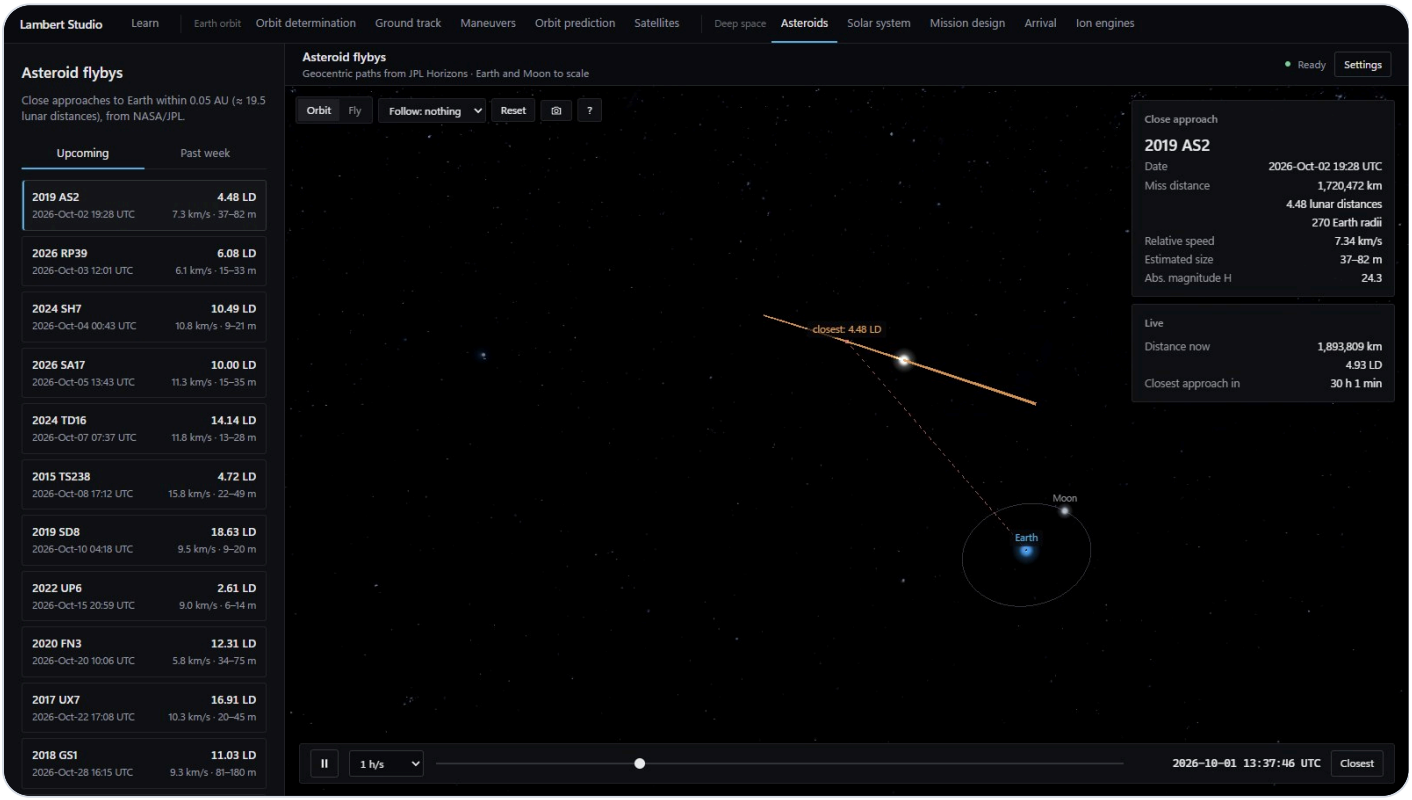
Manoeuvre planner

Hohmann, bi-elliptic and plane-change transfers built from state vectors, with Δv per burn, propellant from the rocket equation and a cost comparison.



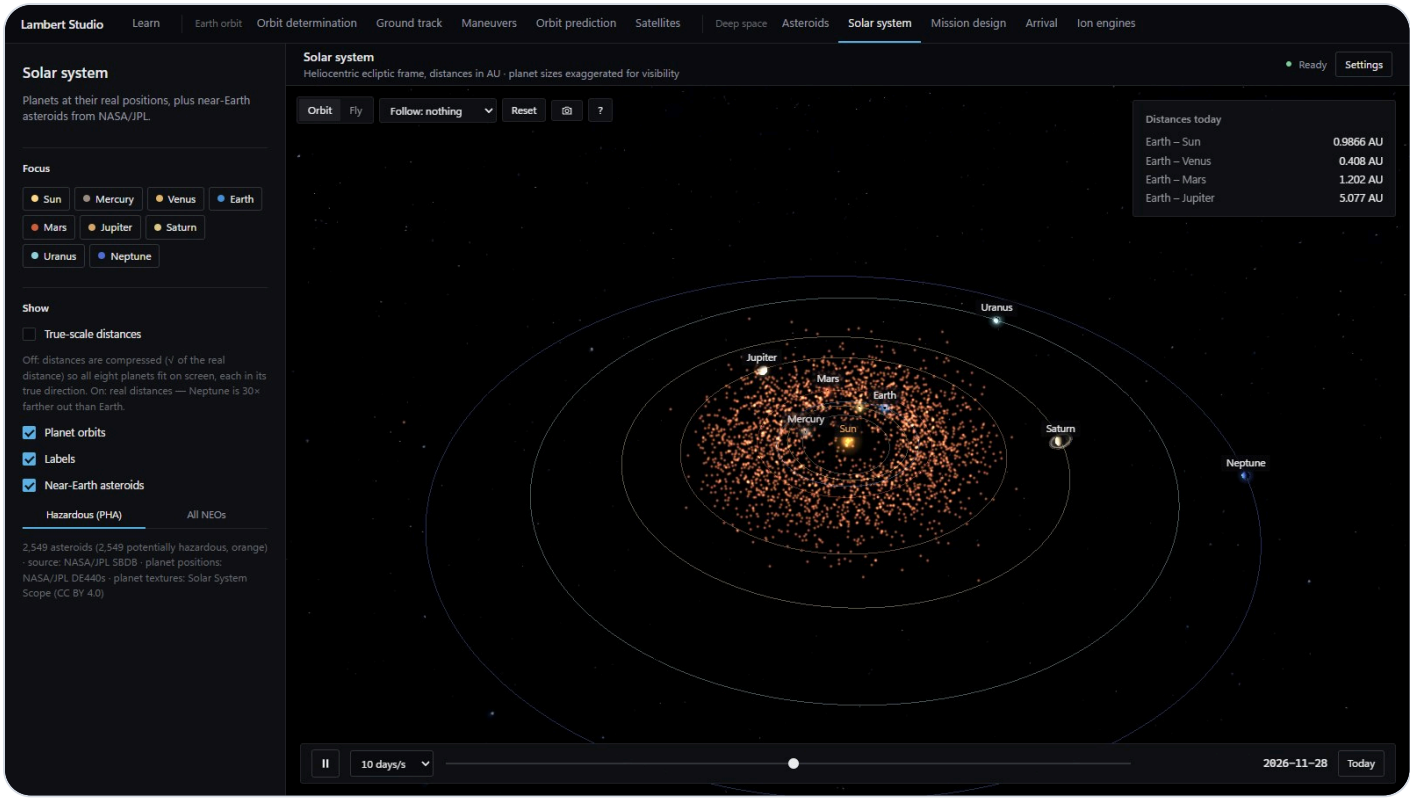
Realistic orbit prediction

Earth's gravity field to degree 6 plus its elliptical equator, the Moon and Sun from JPL, sunlight pressure with Earth's shadow and the NRLMSIS atmosphere, shown here on a geostationary satellite over one year.



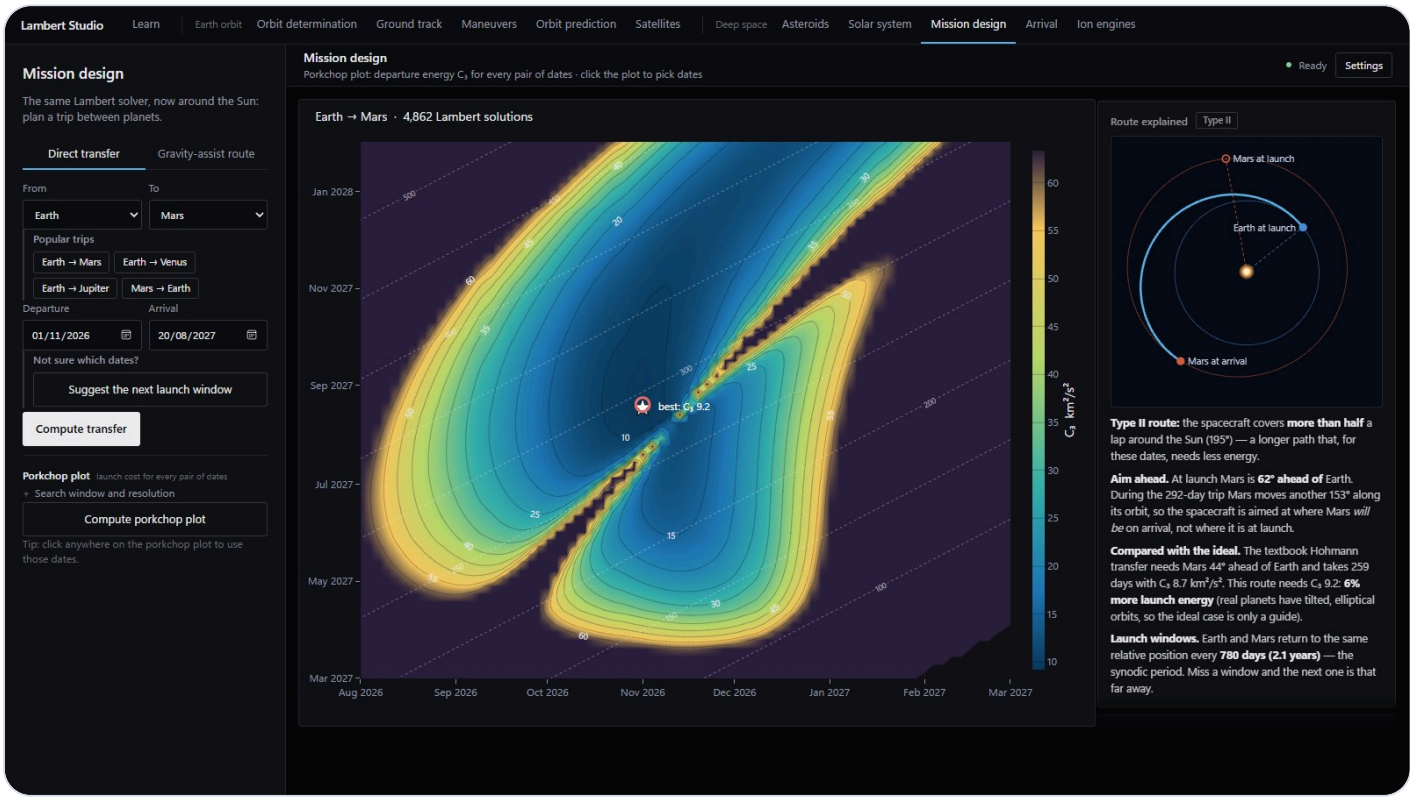
Asteroid flybys

Upcoming close approaches from NASA/JPL, each drawn with its precise path from JPL Horizons, to scale with Earth and the Moon.



Solar system

Planets from the JPL DE440s ephemeris and 2,549 potentially hazardous asteroids, with compressed distances so all eight planets fit on screen.



Mission design

Porkchop plots from thousands of Lambert solutions, a plain-language route explanation and a search for the next launch window.



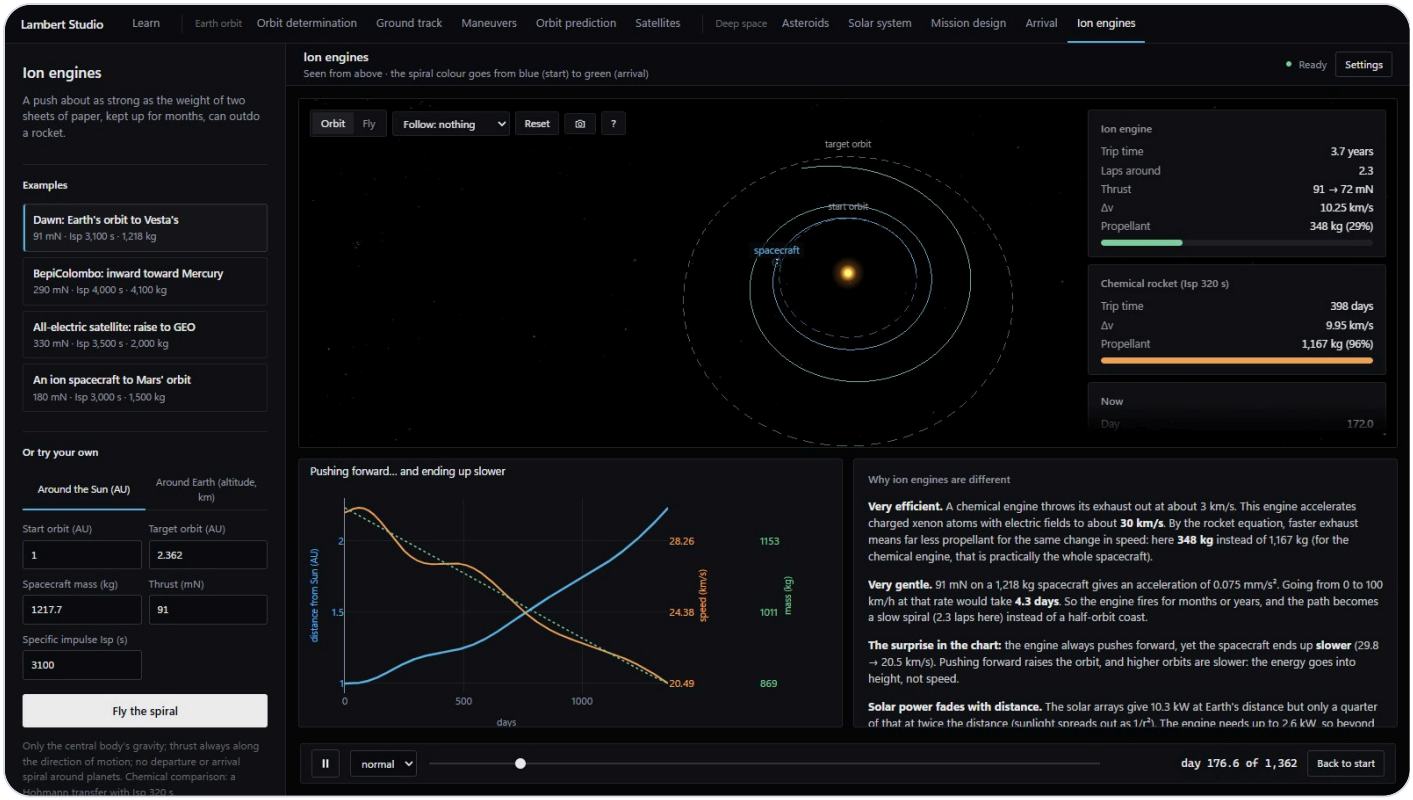
Gravity-assist routes

Up to four flybys, including repeat visits to the same planet, optimised automatically and explained step by step, with the speed gained at each flyby.



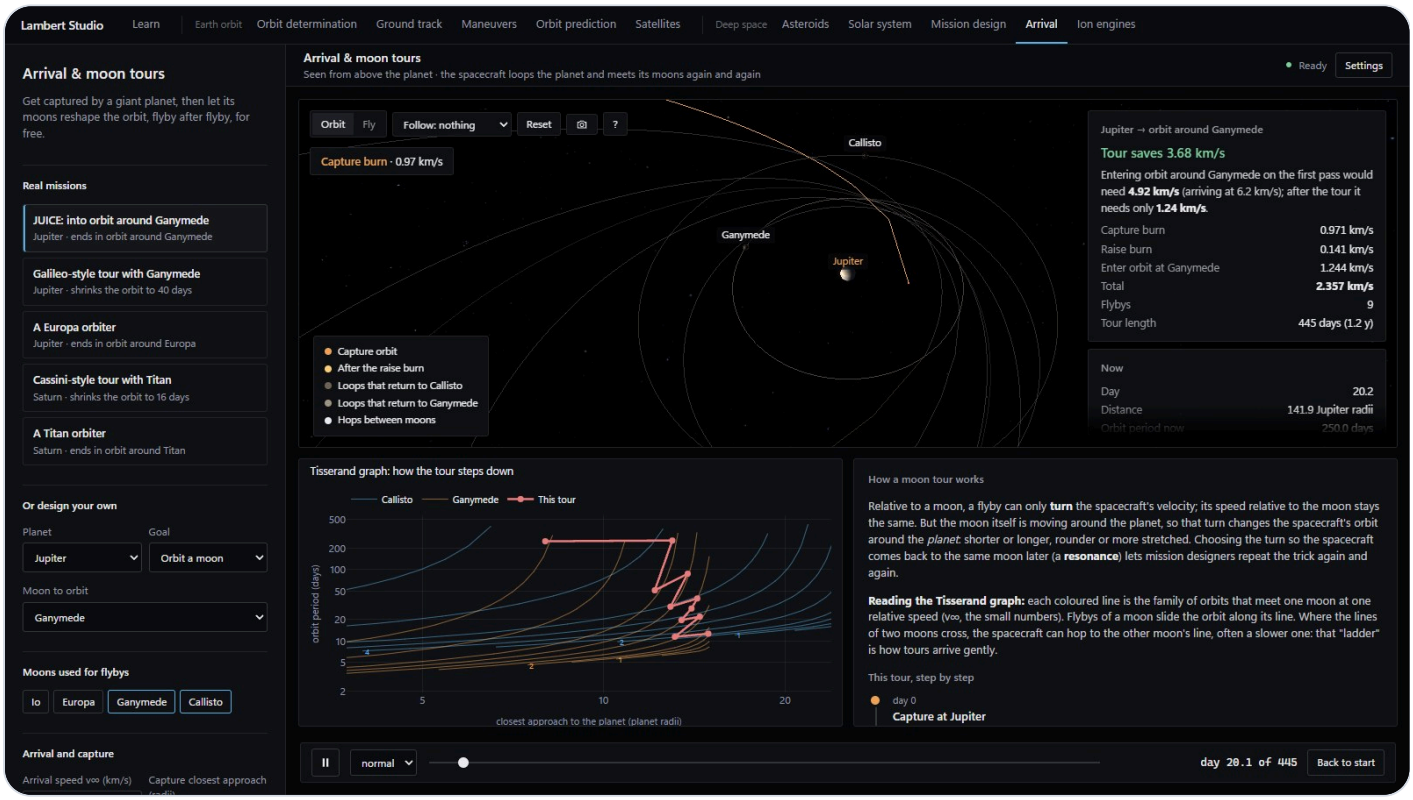
Full-gravity check

Any designed route is flown again with every planet and the Moon pulling at once. Voyager's design would have crashed into Jupiter; 4 m/s of corrections fix it.



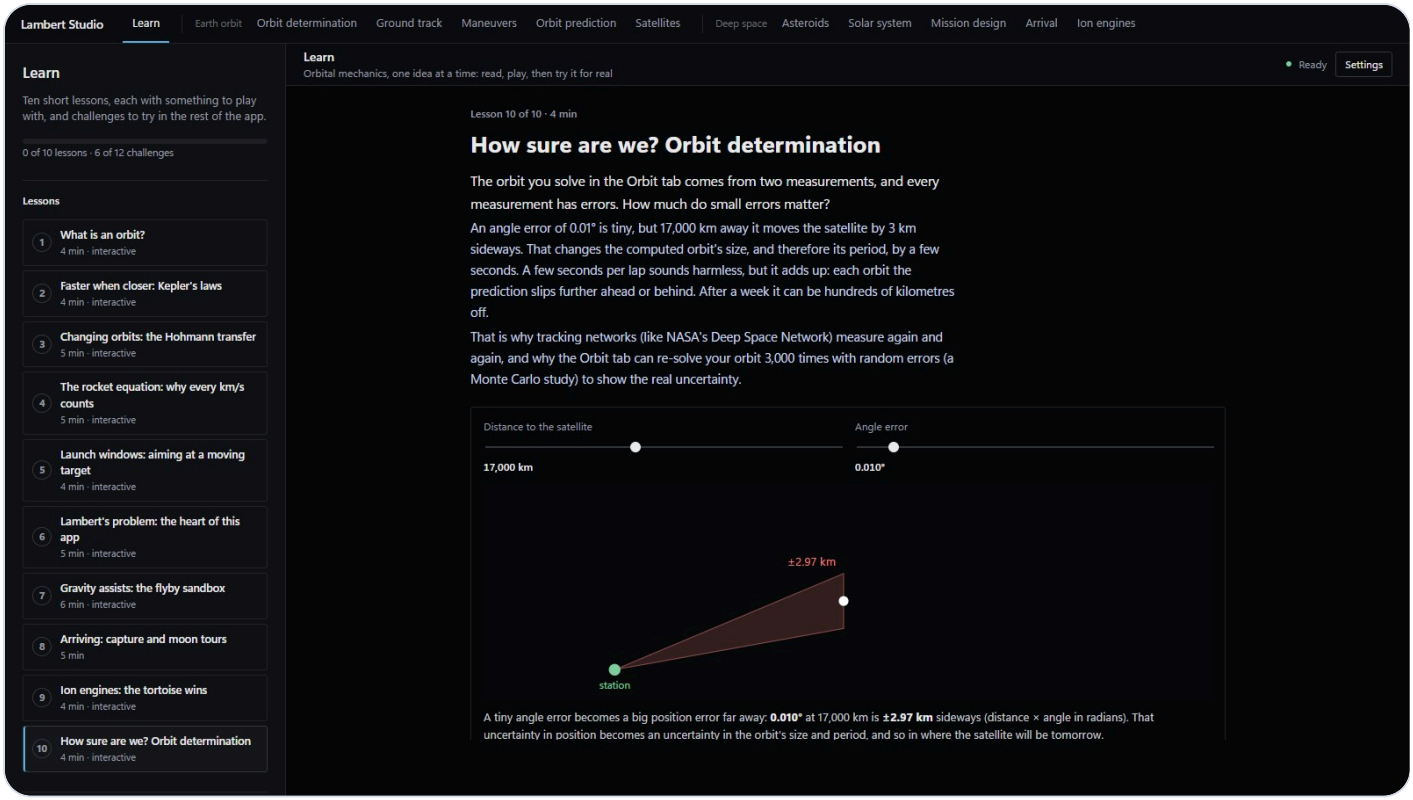
Ion engines

Low-thrust spirals with solar-power-limited thrust: Dawn's engine loses about a fifth of its push between Earth and Vesta.



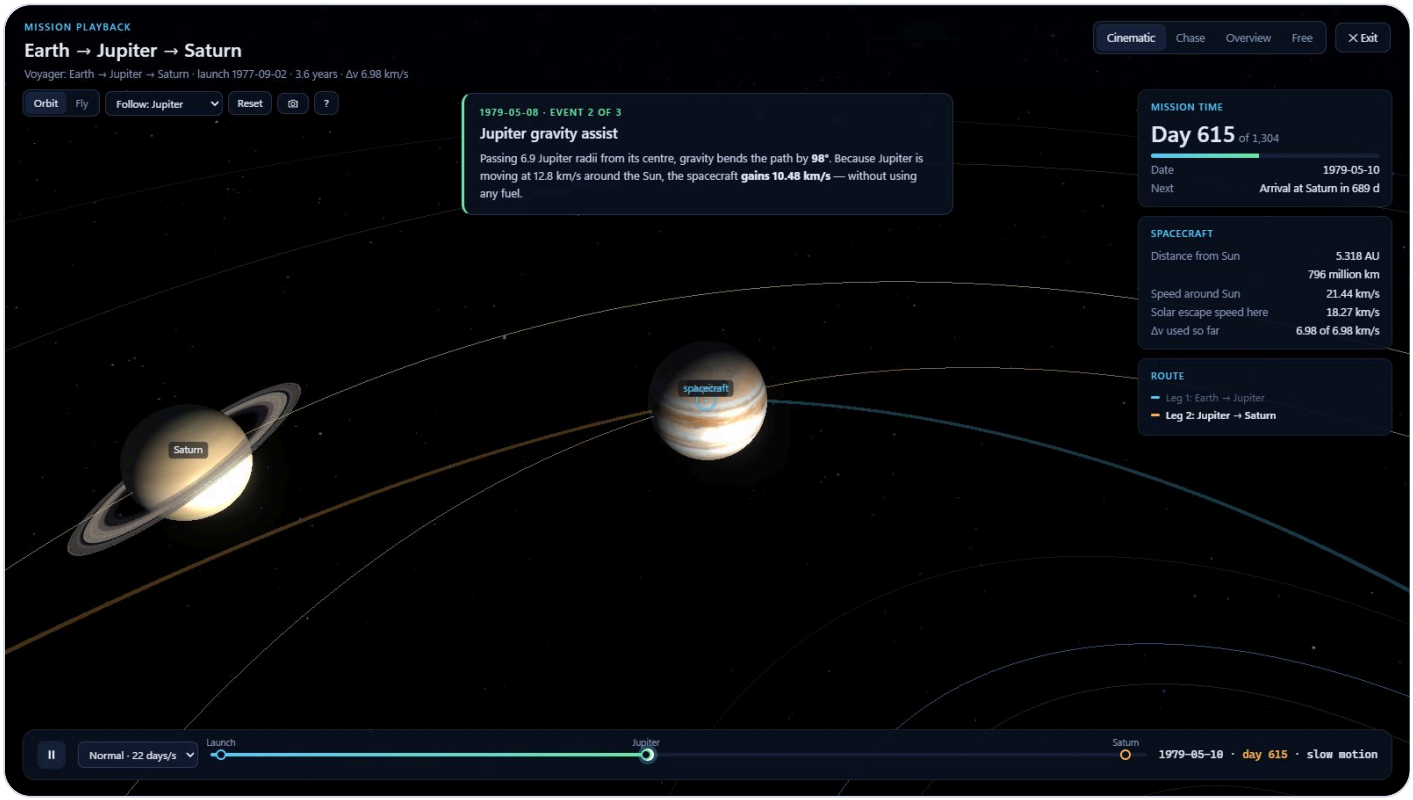
Arrival and moon tours

Capture burns at the giant planets and Tisserand-graph moon tours, the method behind Cassini and Europa Clipper.



Learn

Ten interactive lessons and twelve challenges that check themselves as you use the app.



Mission playback, Voyager at Jupiter (May 1979). The cinematic camera swings to Jupiter and time slows down. Passing 6.9 Jupiter radii from its centre bends the path by 98° and adds 10.48 km/s, enough to leave the Solar System.

T+2

HOW IT WAS BUILT

From a script to a studio, one verified step at a time

The project grew in stages. Each stage had to pass the tests from every earlier stage before the next began.

PHASE 1 · FOUNDATION

Rebuild and verify the solver

Time systems, coordinate frames, a universal-variable Lambert solver and orbital elements, checked against textbook worked examples and physical invariants: energy and angular momentum conserved to 10^{-15}, and propagation landing on the second observation to within 10^{-11} km.

PHASE 2 · APPLICATION

A desktop app with a web-grade 3D engine

Python keeps the physics and serves a small JSON API on the local machine; the interface is drawn with Three.js (WebGL) and Plotly. After the embedded Qt browser crashed on this machine, the window moved to Microsoft Edge's app mode, with crash logging and automatic recovery.

PHASE 3 · REAL DATA

Satellites, asteroids and the planets

CelesTrak element sets for thousands of satellites, NASA/JPL close approaches and Horizons trajectories, and the DE440s ephemeris via Skyfield, with offline caching so the app keeps working without a connection.

PHASE 4 · ENGINEERING TOOLS

Manoeuvres and perturbations

A manoeuvre planner that derives every burn from state vectors, and a numerical integrator (SciPy DOP853) with J2 and an exponential atmosphere, checked against the analytic J2 drift rates.

PHASE 5 · MISSION DESIGN

Gravity assists and multi-revolution Lambert

A vectorised Lambert solver (10× faster, identical to 10^{-11} km/s) with multi-revolution solutions, a beam search over launch dates and flight times, and patched-conic flyby physics. Widening the search took Cassini's route from 9.7 to 6.9 km/s and onto its historical dates.

PHASE 6 · EXPERIENCE

Mission playback, free camera, guided inputs

A full-screen, cinematic playback of any mission; a fly-anywhere camera; and recommended values with sources: NASA Deep Space Network stations, flown engines, real spacecraft masses and a launch-window finder.

PHASE 7 · DELIVERY

Tests, packaging and the course hand-in

A pytest suite covering the physics and the API, browser-driven interface tests, a stand-alone Windows build with PyInstaller, and a generated single-file version of the solver that meets the assignment's format.

PHASE 8 · DEPTH

Learning, arrival, ion engines, uncertainty

A Learn tab with lessons and self-checking challenges, capture and moon-tour design, low-thrust spirals, a deep-space burn optimiser (the Juno trick) and Monte Carlo error bars on the solved orbit.

PHASE 9 · FIDELITY

A more accurate simulation

The force model and the route model were rebuilt to be much closer to reality, and every step was measured against known values (see the next section).

T+3

ARCHITECTURE

Physics in Python, pictures in the browser

DATA

Sources

- NASA/JPL DE440s ephemeris
- JPL Horizons, SBDB, CNEOS
- CelesTrak GP element sets
- NASA Blue & Black Marble

PYTHON

Astrodynamics core

- Lambert (single & multi-rev, batch)
- Frames, time, elements
- Manoeuvres, high-fidelity force model
- Full-gravity route check (n-body)
- Patched-conic route optimiser

LOCAL API

studio_server

- JSON endpoints on 127.0.0.1
- Input validation, clear errors
- Request logging, caching

INTERFACE

Web UI in a desktop window

- Three.js WebGL, bloom, shaders
- Plotly charts
- Edge app mode (Qt fallback)

`data → Python core → JSON API → Three.js / Plotly → desktop window`

T+4

MORE ACCURATE

From textbook models to the real force field

Two simplifications limited the first version: Earth's orbit prediction knew only J2 and an exponential atmosphere, and gravity-assist routes assumed one planet pulls at a time. Both were replaced, and each change was checked against a known value.

UPGRADE	BEFORE	NOW
Earth's gravity	J2 bulge only	EGM96 to degree 6, plus the elliptical equator (C22, S22)
Other bodies	none	Sun and Moon positions from JPL DE440s
Sunlight	none	Radiation pressure with a conical Earth shadow (umbra and penumbra)
Atmosphere	exponential	NRLMSIS 2.1, following solar activity (F10.7) and the day/night cycle

UPGRADE	BEFORE	NOW
Routes	patched conics	Flown again with the Sun, eight planets and the Moon, corrected in the B-plane like a real navigation team
Ion engines	constant thrust	Thrust limited by sunlight, falling as $1/r^2$

Geostationary check. With these forces a satellite parked at 45°E drifts toward the stable point near 75°E and its orbit tilts 0.95° in a year. Real operators see about 0.75–0.95° per year, which is why they spend fuel on north–south station keeping.

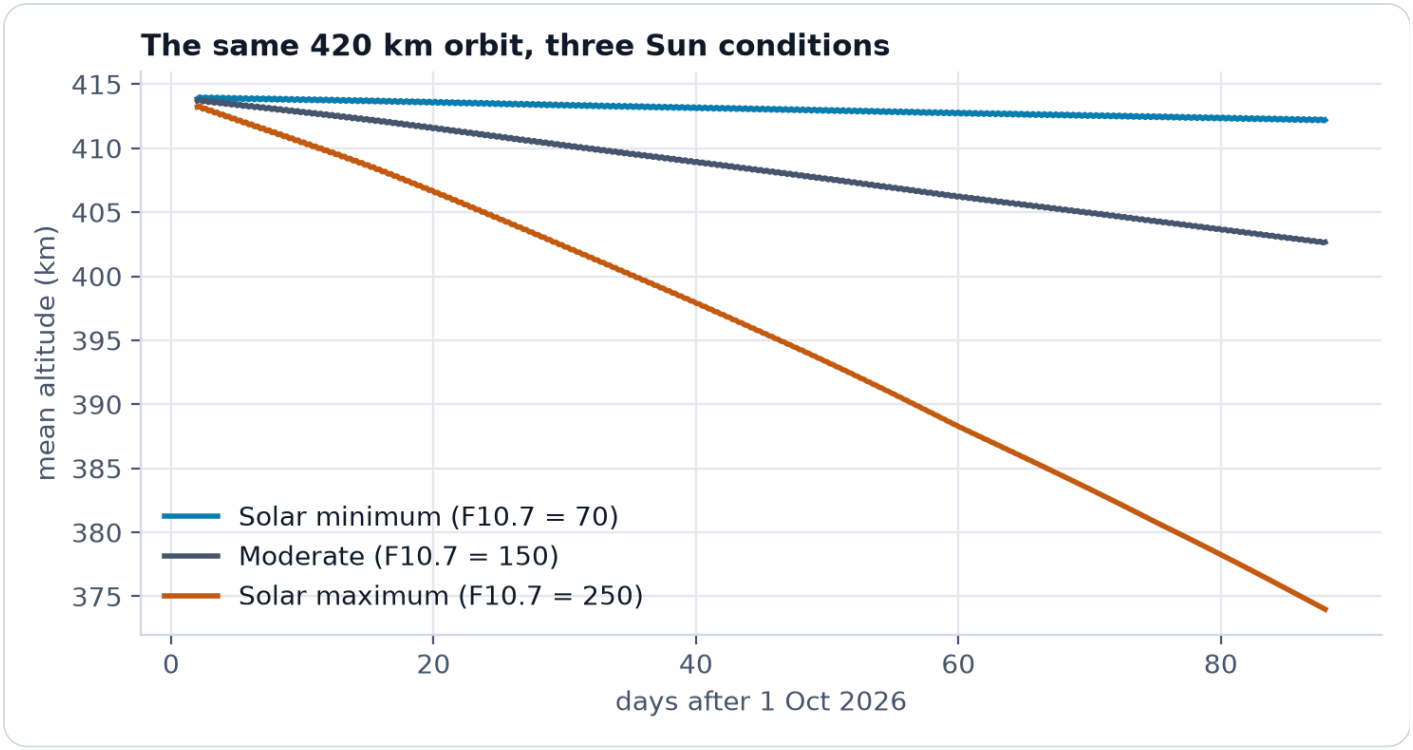
Route check. Flown uncorrected in real gravity, the simple-model Mars trip misses by about 641 thousand km and Voyager crashes into Jupiter. After the corrections, Mars arrival altitude agrees to within 10 km and Voyager's Saturn altitude to within 50 km.

T+5

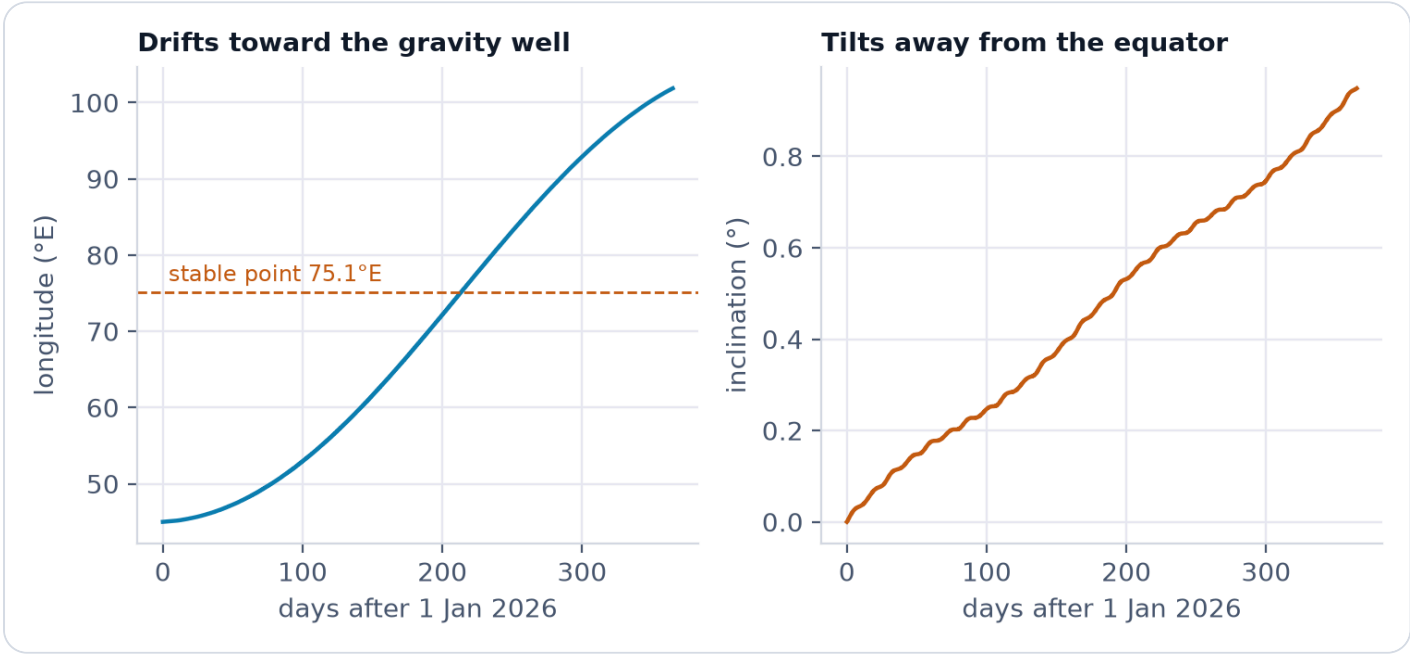
OVER TIME

How the answers change with time

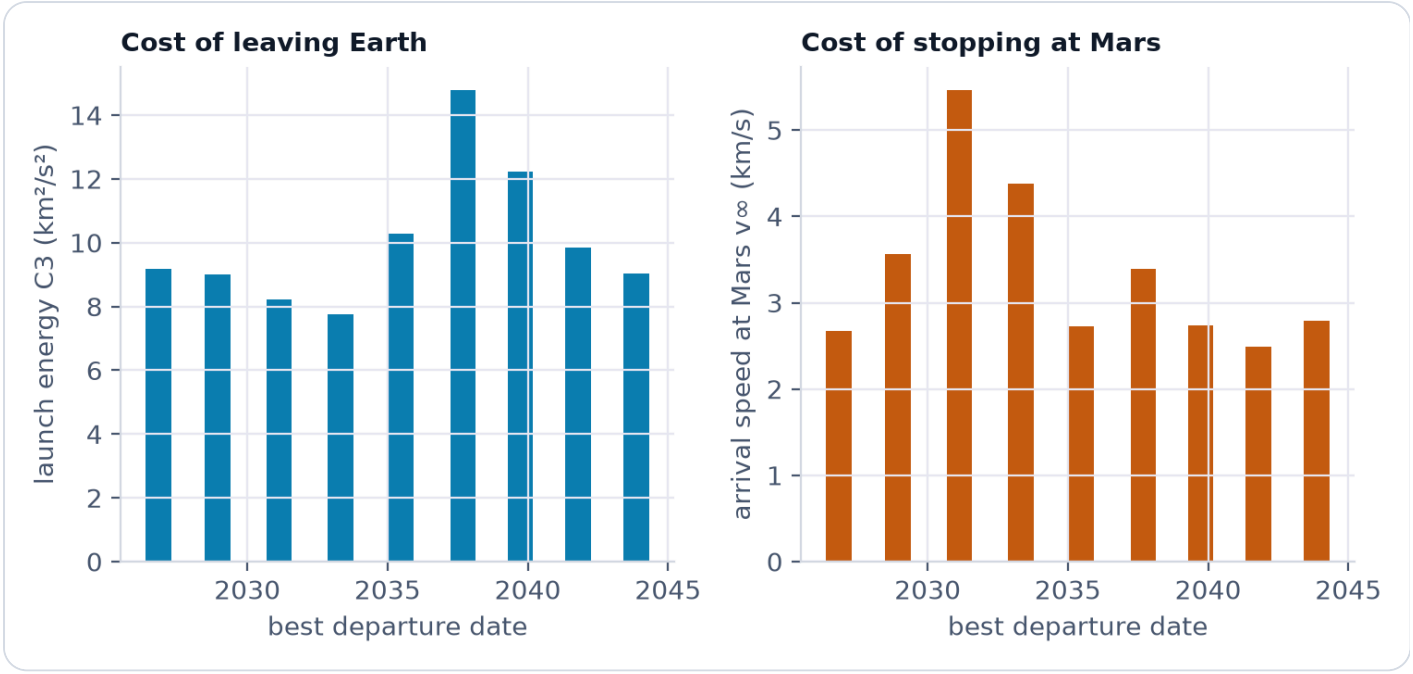
A simulation does not give one answer, it gives an answer *for a date*. The same orbit decays 23 times faster at solar maximum than at solar minimum, the same Mars trip costs nearly twice the launch energy in one window than in another, and a satellite that is parked today drifts tomorrow. Five studies, all run with the app's own physics, show how.



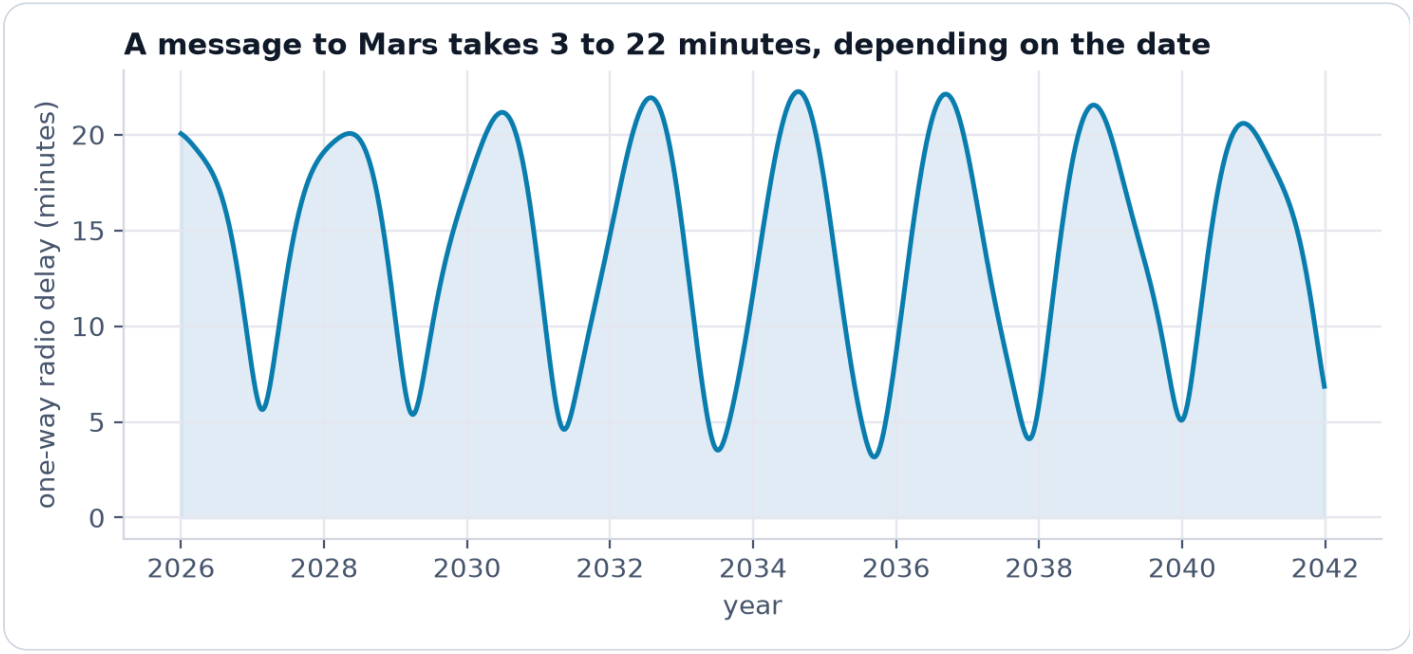
1 · The Sun sets the decay rate. A 420 km orbit loses 20 m/day at solar minimum, 131 m/day at moderate activity and 461 m/day at solar maximum, a factor of 23. After 90 days the orbit is 413, 403 or 373 km high. (Fixed spacecraft area and mass, no reboosts.)



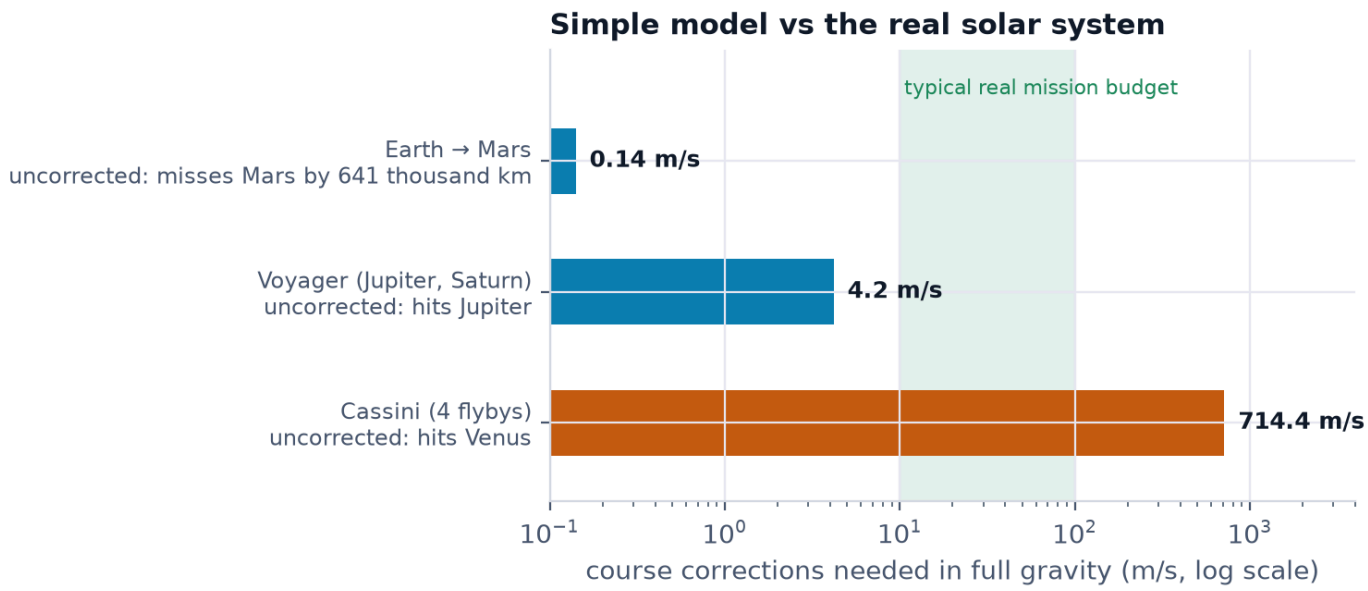
2 · A parked satellite does not stay parked. Starting at 45°E the satellite slides east, passes the stable point at 75°E and reaches 102°E after a year, while the Moon and Sun tilt its orbit by 0.95°. Without station-keeping burns the slot is lost.



3 · The same trip, nine launch windows. The cheapest Earth–Mars transfer repeats every 26 months, but its cost does not. Launch energy ranges from 7.8 to 14.8 km²/s², the arrival speed from 2.5 to 5.5 km/s, and the flight time from 195 to 400 days, because Mars's orbit is elliptical. First window: launch 2026-10-31, arrive 2027-08-22.



4 · Distance changes the conversation. Mars is 57 million km away at its closest and 400 million km at its farthest: a radio message takes 3.2 to 22.2 minutes one way, so a rover cannot be joysticked.



5 · Time amplifies small errors. Every route is exact in the simple model and wrong in real gravity, and the error grows along the trip. Corrections of 0.1 m/s (Mars), 4 m/s (Voyager) and 714 m/s (Cassini, whose design burns during flybys) keep them on target.

Shelf life of each answer

OUTPUT	WHAT CHANGES IT	HOW LONG IT STAYS VALID
Orbit from two sightings	Measurement error, then perturbations	Exact at the sighting times; the error bars grow as the orbit is propagated
Hohmann and Lambert transfers	Geometry of the two bodies	Fixed for a given pair of states; the *best date* repeats every synodic period
Satellite positions (CelesTrak)	Drag, manoeuvres, new element sets	Best at the element-set date; low orbits go stale within days
Decay and lifetime	Solar activity, 11-year cycle	Only as good as the solar-activity forecast: a factor of 23 between minimum and maximum
Planet positions (DE440s)	Nothing, for practical purposes	Decades, with errors of a few kilometres
Gravity-assist route	Planet alignment	Good windows come back every few years to decades (Voyager's alignment: once in about 175 years)

> **Reading these results honestly.** The decay and geostationary studies use a fixed spacecraft and no station-keeping, so they show how the *environment* changes the answer, not what a real operator would do. Window costs are the lowest launch energy found in a grid search around each window. Everything is regenerated by one script (`report/study_time.py`), so the figures can be refreshed for any date.

T+6

VALIDATION

Checked against textbooks and against history

The route optimiser is never told the real dates. Given only the planets, a launch period and a trip-time limit, it finds the cheapest trajectory, and the answers land on the historical missions.

CHECK	LAMBERT STUDIO	REFERENCE	DIFFERENCE
TEXTBOOK AND PHYSICS			
Curtis Example 5.2, Lambert velocities	v_1 = (−5.9925, 1.9254, 3.2456) km/s	published	< 4×10^{-5} km/s
Hohmann transfer, 300 km LEO to GEO	3.8926 km/s	3.893 km/s	< 0.001 km/s
Sun-synchronous orbit, node drift (integrated)	0.990 °/day	0.9856 °/day	0.4 %
Geostationary orbit tilt growth (Sun and Moon)	0.95 °/year	0.75–0.95 °/year	in range
Force-model gradient vs potential (J2–J6, C22)	finite differences	analytic	agree
Mars arrival altitude, full solar-system gravity	after corrections	planned altitude	< 10 km
Voyager Saturn altitude, full gravity	after 4 m/s of corrections	planned altitude	< 50 km
HISTORICAL MISSIONS, DATES FOUND BY THE OPTIMISER			
Mariner 10, Venus flyby	6 Feb 1974	5 Feb 1974	1.6 days
Voyager, launch	2 Sep 1977	5 Sep 1977	2.2 days
Galileo, Venus flyby	21 Feb 1990	10 Feb 1990	11 days
Cassini, launch	16 Oct 1997	15 Oct 1997	1.4 days
Cassini, Earth flyby	17 Aug 1999	18 Aug 1999	4 hours

In total: 136 automated checks and 58 pytest tests, plus browser-driven tests that press keys, drag the camera and play missions to confirm the interface behaves as designed.

T+7

LESSONS

What building it taught

Test the behaviour, not the screenshot

A page can look right and still be wrong. Driving a real browser to press keys, double-click and play missions caught bugs that screenshots never showed.

Search width changes the answer

Judged leg by leg, a route that looks expensive early can win later. A 10× wider beam search turned Cassini from worse-than-direct into the historical route.

Know where the maths breaks

Galileo's Earth-to-Earth loop took exactly two years, so start and end points coincide and Lambert's problem is undefined. The app flies a nearby loop and says so.

The platform is part of the system

An embedded browser crashing in native code, and Windows silently redirecting where Python saves files, were fixed by measuring first and changing architecture second.

More physics is not automatically better

A correction that looks right can be wrong in kind: osculating orbital elements wobble by 10 km from J2 alone, so decay had to be measured from orbit averages. Each new force was added only after it reproduced a known number.

Real navigation is iteration

A route designed in simplified physics cannot be flown as is. Navigators aim, fly, measure and correct; the app does the same with small trajectory-correction burns and reports honestly when a design needs large ones.

T+8

STACK & LIMITS

Tools, and what the model leaves out

`Python 3.13` `NumPy` `SciPy · DOP853` `Skyfield · JPL DE440s` `NRLMSIS 2.1` `EGM96` `sgp4` `Three.js · WebGL` `Plotly.js` `Microsoft Edge app mode` `PySide6` `pytest` `PyInstaller` `git`

Model limits

- Route design is still patched conics; the full-gravity check corrects it but cannot redesign it. Designs that burn during flybys (Cassini) need large corrections.
- Earth orbit: gravity to degree 6, no tides or relativity; a single-area "cannonball" spacecraft; full-fidelity runs are slow for low orbits (about 8 s per simulated month).
- Exact resonant loops (start = end) are approximated.
- The assignment's station uses a spherical Earth, as specified.

Next steps

- Redesign routes directly in full gravity.
- Orbit determination from many noisy observations (least squares).
- Satellite pass predictions for any location.
- Continuous integration for the test suite.

Data: NASA/JPL (DE440s, Horizons, SBDB, CNEOS) · CelesTrak · NASA Blue Marble & Black Marble · Solar System Scope textures (CC BY 4.0). **Method:** H. D. Curtis, *Orbital Mechanics for Engineering Students*; D. A. Vallado, *Fundamentals of Astrodynamics and Applications*.

Developed with Claude Code (Anthropic) as an AI pair-programming assistant, with every step directed, run and checked by the author.